

LockPusher: State-Isolated World Models for Pushing under Joint Locks

Anonymous Authors

LockPusher Adapt object predictions. Preserve the robot forecast.

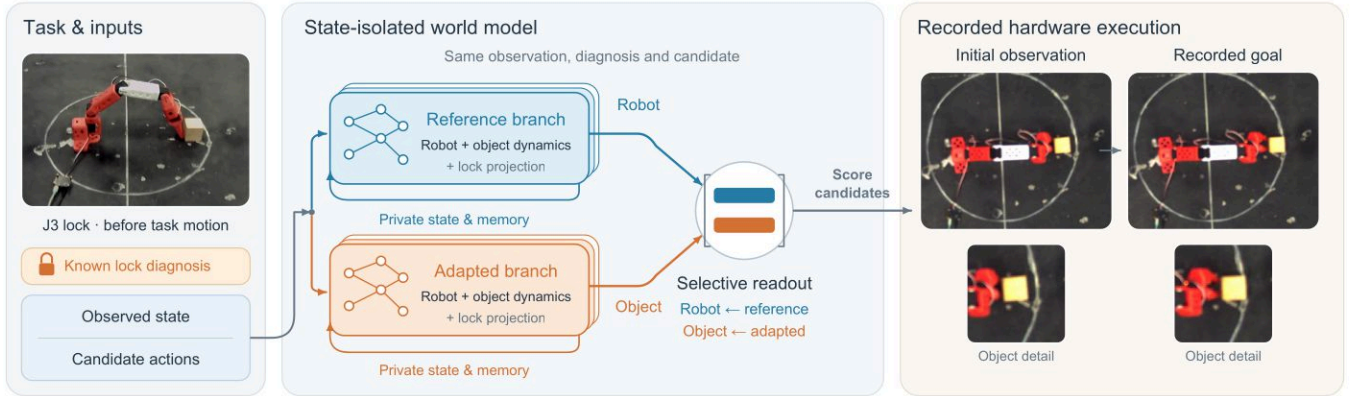


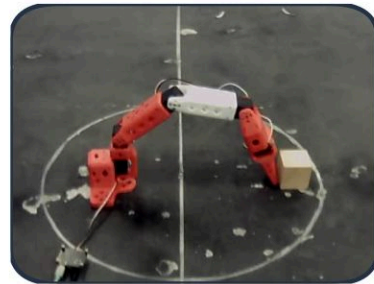
Fig. 1. Overview of LockPusher’s core idea and hardware execution. Given observed robot and object states, a diagnosed joint lock, and an object goal position, LockPusher selects contact actions to reposition the object without grasping. Left: task inputs and physical platform. Center: separate recurrent branches adapt object dynamics while preserving the reference robot forecast, combining both predictions to evaluate candidate pushes. Right: a J3-lock execution illustrates visual replanning toward the goal, with enlarged views showing object displacement.

Abstract—Nonprehensile pushing enables robots to reposition objects that are difficult to grasp, but contact prediction becomes harder after a joint lock. Learned world models can adapt object dynamics from interaction data. In coupled recurrent models, however, an object correction may also change subsequent robot and pusher forecasts. We propose LockPusher, a state-isolated world model for diagnosed joint locks. An analytic projection imposes the lock constraint, while reference and intervention branches maintain separate states and memories and supply robot and object forecasts, respectively. For matched initial conditions and actions, robot and pusher forecasts remain equal to the projected reference at every deterministic rollout step. In a matched simulation benchmark, LockPusher achieves 1.58 mm object-position RMSE and 76.53% task success, reducing prediction error by 46.98% and improving success by 11.24 percentage points relative to the all-state residual baseline. Its advantage persists across the evaluated adaptation-data fractions. Historical paired ablations retain a 16.74% mean reduction in composite object-state RMSE while preserving the reference forecast. On physical hardware, visual replanning completes all 18 trials across the intact arm and five single-joint locks, with a mean object endpoint error of approximately 1.64 pixels for a 40-pixel target displacement.

I. INTRODUCTION

Nonprehensile manipulation extends a robot’s ability beyond pick-and-place by moving objects that are difficult, undesirable, or impossible to grasp. Pushing is particularly useful in cluttered or constrained workspaces, but it is also contact rich: small errors in contact location, friction, or object response can accumulate over a rollout and change which candidate action appears most promising [1], [2]. Model-based planning

**Adapt the object.
Preserve the robot forecast.**



J3-lock hardware trial

Robot prediction



**Preserve
Object response**



Adapt

Fig. 2. Two key challenges addressed by LockPusher. During adaptation, the model must learn how a joint lock changes the object response to contact (object adaptation). During planning, it must retain the reference robot forecast while using the adapted object prediction to select candidate pushes (reference-preserving planning).

addresses long-horizon decision making by predicting these outcomes before execution. Its usefulness therefore depends on a forward model that is accurate enough for action ranking and that respects the robot’s available motion.

The overall task setting and the LockPusher execution interface are shown in Fig. 1. This requirement becomes harder after a joint lock. A diagnosed lock supplies valuable

information—the affected joint and its fixed angle—but it also changes the kinematic and dynamic context in which contact occurs. The robot must continue planning with fewer degrees of freedom, while the response of the pushed object remains uncertain. An analytic model can impose the known constraint, yet accurate contact prediction still depends on quantities such as friction and compliance that are difficult to specify. Learned world models reduce this burden by fitting interaction outcomes from data [2], [3]; damage-recovery methods instead adapt a policy or select compensatory behaviors [4], [5]. These two directions motivate a world model that can incorporate a diagnosed failure and adapt the uncertain object response used for planning.

The two design challenges addressed by LockPusher are summarized in Fig. 2. Existing work improves different parts of this problem. Residual models correct approximate physics, and ActivePusher combines residual physics with uncertainty-guided data acquisition and planning [6]. Dynamics-adaptive policies condition actions on interaction history to handle changing object and surface properties [7]. Fault-aware policies and behavior repertoires can recover task performance under disturbances or damage [4], [5]. Our concern is complementary: once an object correction is introduced into a recurrent world model, which predicted coordinates and internal memories are allowed to change?

This scope matters because robot and object predictions are coupled during rollout. The predicted robot state determines the pusher pose and may also be fed to a controller that generates the next candidate command. Consequently, an object correction fed back through a shared recurrence can change later robot states, pusher motion, and ultimately the action being scored. Restricting the immediate residual output to object coordinates does not remove this indirect path. Conversely, projecting the locked joint to its diagnosed angle guarantees the lock constraint but does not guarantee that the remaining robot forecast matches a chosen reference model. Our archived coupled-model evaluations expose this distinction: object-state error can decrease while free-state and pusher errors increase, even when diagnosed-lock violation is zero (Table 2).

We therefore formulate adaptation as an interface contract: update the object response while retaining a specified robot forecast for the same observation and candidate action. LockPusher combines analytic lock projection with state-isolated prediction. A reference branch, called the carrier, and an intervention branch receive the same projected inputs but maintain separate physical states and recurrent memories. The returned prediction takes robot coordinates from the carrier and object coordinates from the intervention branch, and this combined output is not fed back into either recurrence. For matched initial conditions and action sequences, the construction preserves the projected carrier robot and pusher trajectories at every deterministic rollout step while leaving the intervention branch free to model robot–object interaction and adapt the object forecast.

The adapted object predictions guide candidate selection:

the planner ranks pushes by predicted terminal distance, executes part of the selected command, observes the scene again, and replans. We evaluate prediction and planning in a matched simulation benchmark, examine selective readout and recurrence in historical ablations, and test visual replanning on physical hardware. LockPusher achieves 76.53% simulation success, exceeding Global by 11.24 percentage points, and completes all 18 hardware trials with a mean object endpoint error of approximately 1.64 pixels. The ablations separately test preservation of the reference forecast.

Our contributions are:

- A projected, state-isolated world model that permits object adaptation while preserving a specified robot forecast for matched inputs and action sequences.
- Paired historical comparisons that separate object-prediction gains from collateral robot-prediction changes, together with a recurrence ablation of reference preservation.
- A visual replanning system evaluated against two baselines in matched simulation and across 18 intact-arm and joint-lock hardware trials.

II. RELATED WORK

What changes during adaptation? Prior work adapts different parts of the robot system: behaviour repertoires and robust policies compensate for damage or disturbances [4], [5]; online factor estimation and control-oriented meta-learning support adaptation to changed dynamics [8], [9]; simulator tuning corrects physical parameters [10]. For pushing, recurrent and combined-prediction models update the object response from interaction history [11]–[13]. Building on these adaptation strategies, we impose an explicit output constraint: object corrections must retain a designated robot forecast for the same candidate action.

How should contact dynamics be represented? Pushing depends on friction, geometry, contact location, and speed [14]. Analytical sliding models [15] and data-efficient learned predictors [2], [3] offer complementary structure. Residual physics combines a nominal model with learned corrections in pushing and throwing [16], [17]; ActivePusher further links these corrections to active sampling and uncertainty-aware planning [6]. Relational models encode interacting entities [18], [19], with material adaptation in AdaptiGraph [20] and physical structure in PIN-WM [21]. LockPusher addresses a different design choice: which coordinates an object correction may change, and how that correction enters subsequent recurrence.

How are predictions used for manipulation? Visual foresight and neural dynamics support action evaluation and model-based learning [1], [22]. DyWA couples future-state and action prediction with history-based dynamics adaptation [7]; structured world models learn affordance dynamics from human video [23]. At execution time, hybrid MPC, online pushing models, and object-centric planning address contact modes and rearrangement [24]–[26], while HACMan learns contact locations and motions [27]. Together with

ActivePusher [6], these works show how improved predictions guide manipulation. Our contribution concerns the prediction interface: reference robot and adapted object coordinates are combined for scoring, while each branch retains its own recurrent state. These methods establish the research context; our numerical comparison uses matched diagnosed-lock tasks and inputs.

III. PROBLEM FORMULATION

Planning under a diagnosed lock. Let $s = [r, o] \in \mathcal{S}$ denote the joint robot–object state, where $r = [q, \dot{q}]$ is the robot state and o is the object state. The robot receives commands $a \in \mathcal{A}$. A diagnosis $d = (m, \bar{q})$ specifies a binary joint-lock mask and the corresponding fixed angles. For a fixed diagnosis, we write the unknown discrete-time dynamics as

$$s_{t+1} = f_d^*(s_t, a_t). \quad (1)$$

The diagnosis induces admissible states $\mathcal{S}_d$ and commands $\mathcal{A}_d$: locked joints remain at $\bar{q}$ with zero velocity and receive no command. Given an initial state s_0 and object goal region $\mathcal{G}$, the planning problem is to find a sequence $a_{0:H-1} \in \mathcal{A}_d^H$ whose trajectory remains admissible and reaches $o_H \in \mathcal{G}$. We study planar pushing with diagnosis available before planning. Contact dynamics are unknown, so candidate outcomes must be predicted from learned models [2], [11].

Object-dynamics adaptation. Let $\mathcal{D}$ collect observed interaction trajectories, each containing states $s_{0:H}$, actions $a_{0:H-1}$, and diagnosis d . A predictor uses these observations to approximate the outcomes of candidate actions. We distinguish the robot forecast from the object response because adaptation of a coupled recurrent model can change both, even when only the object prediction is intended to improve. Given a designated reference predictor F_c , our learning objective is to reduce object prediction error while preserving its robot forecast. For the same initial observation, diagnosis, and action sequence, the adapted predictor must satisfy

$$\hat{r}_t = \hat{r}_t^c, \quad t = 1, \dots, H, \quad (2)$$

where $\hat{r}_t^c$ denotes the reference robot prediction after imposing the diagnosed constraint. This architecture-independent requirement covers both locked and free joints throughout the rollout.

Planning with a preserved robot forecast. For each candidate k , the adapted model returns a trajectory $\hat{s}_{1:H}^{(k)}$ and terminal object position $\hat{p}_H^{(k)}$. Following predictive action evaluation [1], [24], our finite-candidate planner chooses

$$k^* = \arg \min_k \|\hat{p}_H^{(k)} - g\|_2, \quad (3)$$

where g is the target position. Thus, object adaptation may change the ranking of candidates while each retains its reference robot forecast. The preservation requirement specifies equality to F_c ; prediction accuracy and task success are evaluated empirically. We therefore assess object prediction, preservation of the robot forecast, and the outcome of executed actions separately. The method below constructs a predictor that satisfies this requirement by design.

IV. METHOD

LockPusher combines diagnosed constraints, object-dynamics adaptation, and candidate evaluation in a single prediction pipeline, as illustrated in Fig. 3. A reference branch and an adapted branch start from the same observation but maintain separate rollout states. The planner reads robot predictions from the reference and object predictions from the adaptation branch. We describe constraint projection, state isolation, the network implementation, training, and execution in turn.

A. Incorporating the diagnosed lock

LockPusher first incorporates the known constraint into prediction. For each joint j , define

$$\Pi_d(q)_j = (1 - m_j)q_j + m_j\bar{q}_j, \quad (4)$$

$$\Pi_d(\dot{q})_j = (1 - m_j)\dot{q}_j, \quad \Pi_d(a)_j = (1 - m_j)a_j. \quad (5)$$

Object coordinates are unchanged. Applying this projection to model inputs and predicted transitions keeps locked positions fixed and locked velocities zero throughout the rollout. Because m_j is binary, applying the projection twice has the same effect as applying it once. Learned object adaptation therefore operates within an explicitly constrained robot prediction.

B. Why rollout state must be isolated

A one-step output mask does not, by itself, determine the behavior of a recurrent predictor. To see the distinction, consider a coupled transition with robot and object components f_r, f_o and memory h_t . Suppose an adapted object value $\bar{o}_t$ is inserted into the next model input. Even when the current returned robot state is unchanged, the next prediction becomes

$$\bar{r}_{t+1} = f_r(r_t, \bar{o}_t, a_t, h_t). \quad (6)$$

Relative to an unmodified object state o_t , the next robot difference is

$$\Delta r_{t+1} = f_r(r_t, \bar{o}_t, a_t, h_t) - f_r(r_t, o_t, a_t, h_t). \quad (7)$$

Object changes can propagate through both the state and recurrent memory, affecting future robot outputs. The magnitude and effect of this feedback depend on the coupled model.

LockPusher removes this feedback path at the rollout interface. Each branch advances its own state and memory. The combined state supplies candidate evaluation, while recurrence uses the private branch states. Both branches may still contain robot–object interactions internally. Isolation controls state exchange between predictors while retaining robot–object interactions within each branch.

The deployed block-structured model is a special case: its robot transition does not consume object state. In that configuration, object-only updates can already preserve robot predictions when robot parameters and inputs remain fixed. State isolation supplies the explicit interface contract and also applies to historical coupled predictors. Distinguishing these configurations is necessary when interpreting the recurrence comparisons: a preservation test on a coupled model and an object-adaptation test on the deployed architecture answer different questions.

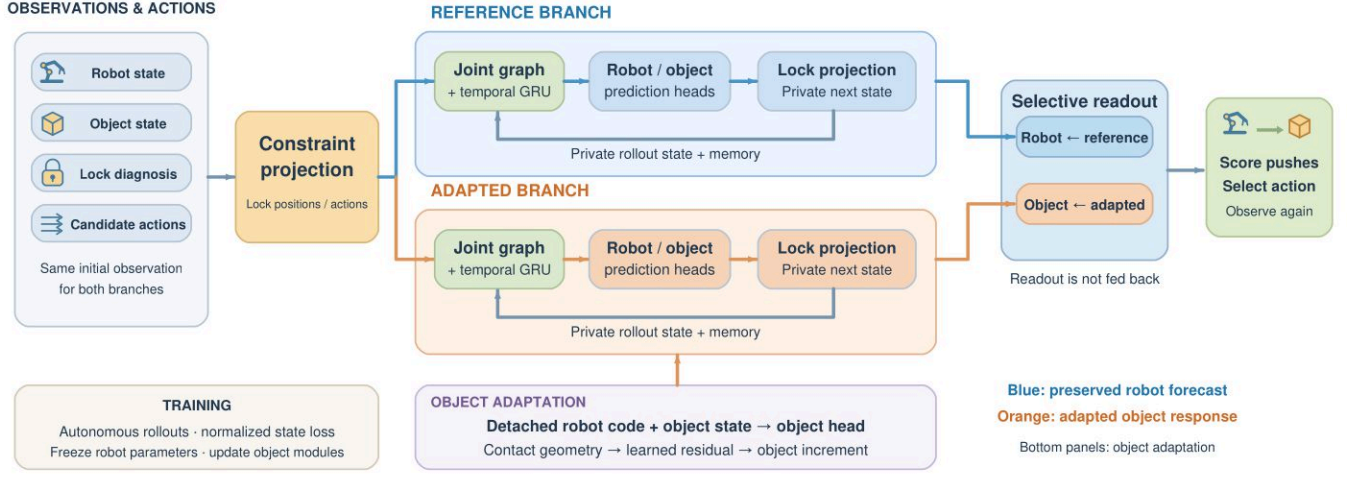


Fig. 3. LockPusher architecture. Constraint projection supplies both recurrent branches, which maintain private states and memories. Selective readout combines reference robot coordinates with adapted object coordinates for candidate scoring, without feedback into either branch. Detached robot features and contact geometry support object adaptation; the planner executes the selected action prefix and replans.

C. Object adaptation with separate rollout states

The carrier F_c supplies the reference prediction. An intervention model F_i supplies the adapted object response. Each branch maintains its own physical state s_t^b and recurrent state h_t^b , where $b \in \{c, i\}$. Starting from the same projected observation, both branches receive the same projected candidate action:

$$(s_{t+1}^b, h_{t+1}^b) = F_b(s_t^b, \Pi_d(a_t), d, h_t^b), \quad (8)$$

$$s_{t+1}^b \leftarrow \Pi_d(s_{t+1}^b), \quad (9)$$

$$\hat{s}_{t+1} = [r_{t+1}^c, o_{t+1}^i]. \quad (10)$$

The returned state combines the two predictions for scoring. Private recurrence preserves the reference rollout while allowing the intervention to propagate its learned object dynamics.

Reference-trajectory preservation. For deterministic branches with matched initial states, carrier parameters, diagnoses, and candidate actions, the returned robot trajectory equals the standalone projected carrier trajectory at every step. The initial carrier states agree. If they agree at step t , the same transition, hidden state, action, and projection produce the same next state. Equation (10) returns that carrier state, establishing the result by induction. The pusher trajectory is also preserved because it is computed from joint positions by the same forward kinematics. For fixed candidates, deterministic evaluation terms computed solely from robot coordinates are also preserved. This property is relative to the chosen carrier; prediction accuracy and joint dynamical consistency are separate properties.

D. Contact-conditioned object adaptation

A joint lock changes the relation between robot motion and object response. We therefore condition object corrections on explicit contact geometry, complementing the learned robot representation [6], [16]. Let z_t denote robot features and

ϕ_t describe pusher motion, relative object geometry, object velocity, and contact proximity. The adapted transition is

$$o_{t+1} = o_t + G_\theta([\text{sg}(z_t), o_t]) + R_\psi(\text{sg}(\phi_t)). \quad (11)$$

The object head G_θ captures the nominal response; R_ψ learns geometry-dependent corrections. Stop-gradient retains the predictive value of robot features while directing adaptation to the object modules. Contact proximity is encoded as $\gamma_{\ell,t} = \sigma((\tau - c_{\ell,t})/\epsilon)$, where $c_{\ell,t}$ is the clearance at rollout stage ℓ , τ is a distance offset, and $\epsilon > 0$ controls smoothness. These features allow the residual to respond to changes in contact configuration. Independent recurrence then propagates the adapted object dynamics while maintaining the reference robot trajectory.

E. Matched adaptation

For the large-scale adaptation study, all variants start from the same pretrained carrier. We freeze its robot parameters and update the object head. LockPusher additionally updates its geometric object residual. The no-residual variant excludes this correction, while the global variant learns corrections to all state coordinates. All variants use the same data and optimization budget.

The underlying adaptation model is trained directly. Its predictions $\tilde{s}_t$ minimize normalized squared state errors over an autonomous prediction window; state isolation is applied when returning candidate predictions:

$$\mathcal{L} = \frac{1}{d_s H} \sum_{t=1}^H \|D^{-1}(\tilde{s}_t - s_t)\|_2^2. \quad (12)$$

The diagonal matrix D normalizes state coordinates. Robot parameters are fixed during object adaptation; the global baseline also adapts robot coordinates.

The scales in D are fixed across variants. The validation-selected model is frozen for closed-loop evaluation.

Algorithm 1: Object-dynamics adaptation

Input: Pretrained model F_0 , training/validation data, scale D , epochs E , horizons H_e, H_v .

Output: Validation-selected adapted model F_i .

-
- 1 Initialize $F_i \leftarrow F_0$; freeze robot parameters.
 - 2 Retain the initial model as a validation candidate.
 - 3 **for** epoch $e = 1, \dots, E$ **do**
 - 4 Set rollout horizon $H \leftarrow H_e$.
 - 5 Sample one contiguous window per training trajectory.
 - 6 Roll out F_i autonomously from each initial state.
 - 7 Update object head and residual using $\mathcal{L}$.
 - 8 Evaluate horizon- H_v object-position MSE.
 - 9 **return** checkpoint with lowest validation MSE.
-

F. Candidate selection and execution

A new observation reinitializes both branches after execution. Simulation executes action commands; hardware moves toward selected joint references. The experiment section reports the candidate counts and execution settings.

For K candidates and horizon H , each branch requires KH transitions. Candidates can be batched independently; each maintains its own temporal recurrence. The two-branch construction adds a reference rollout to the intervention rollout. Preservation applies to each matched candidate. Adaptation can change the selected action through its effect on predicted object outcomes.

Algorithm 2: State-isolated candidate evaluation

Input: Observation s_0 , diagnosis d , goal g , candidates $\mathcal{U}$, branches F_c, F_i .

Output: Selected sequence $a_{0:H-1}^{(k^*)}$.

-
- 1 **for each** candidate $a^{(k)} \in \mathcal{U}$ **do**
 - 2 Initialize both branches at $\Pi_d(s_0)$ with their own initial memories.
 - 3 **for** $t = 0, \dots, H - 1$ **do**
 - 4 $u_t \leftarrow \Pi_d(a_t^{(k)})$
 - 5 Update (s^b, h^b) independently, $b \in \{c, i\}$.
 - 6 Project each state: $s^b \leftarrow \Pi_d(s^b)$.
 - 7 Return $[r^c, o^i]$ without branch feedback.
 - 8 $\hat{c}_k \leftarrow \|\hat{p}_H^{(k)} - g\|_2$
 - 9 **return** sequence indexed by $\arg \min_k \hat{c}_k$.
-

V. EXPERIMENTS

We evaluate three questions: whether restricted adaptation improves object prediction and closed-loop planning across adaptation-data budgets under diagnosed joint locks; whether selective readout and independent recurrence retain object corrections while preserving the reference robot forecast; and whether the controller completes pushes under physical joint locks.

A. Simulation benchmark and quantitative comparison

Task setup. We evaluate planar box pushing under five single-joint locks in MuJoCo [28]. The object has a fixed

Method	Object RMSE (mm)	Goal distance (mm)	Success (%)
Carrier	5.07	3.32	47.84
Global	2.98	2.06	65.29
LockPusher	1.58	1.43	76.53

Table 1. Full-data simulation results averaged over three training seeds. LockPusher achieves lower prediction and goal errors and higher task success than both baselines. RMSE denotes root mean squared error.

orientation. Dynamics settings comprise nominal conditions, doubled damping, 70% actuator gain, and the combination of the latter two. Each trajectory starts from an independent reset with checks on support geometry and joint limits. Training, validation, and test sets contain 50,000, 2,000, and 6,000 trajectories, respectively.

Implementation and training. The robot encoder uses a five-node chain graph [18], [19], hidden width 136, two message-passing rounds and 136-unit GRUs [29]. SiLU [30] networks have widths $6 \rightarrow 136 \rightarrow 136$ (encoder), $272 \rightarrow 136 \rightarrow 136$ (messages), $136 \rightarrow 136 \rightarrow 2$ (joint head), and $140 \rightarrow 136 \rightarrow 4$ (object head). The residual uses a 12-dimensional descriptor and a 16-unit Tanh hidden layer. Its inputs are previous/next pusher positions, their difference, next-pusher-to-object displacement, object velocity, and pre/post-transition contact gates. Capsule radii are 12 and 8 mm; the object box is 40 mm; $\tau = -0.005$ m and $\epsilon = 0.002$ m. The state dimension is $d_s = 14$. Normalization scales are 0.1 rad, 1 rad/s, 0.03 m, and 0.1 m/s for joint position/velocity and object position/velocity. Training uses $E = 10$ epochs, with $H_e = 10$ for the first five and $H_e = 25$ thereafter; each trajectory supplies one random contiguous window per epoch. Validation uses $H_v = 25$, including the initial checkpoint.

Baselines. We compare three adaptation variants of the same pretrained model to evaluate the effect of adaptation scope. Carrier adapts the object head with robot parameters fixed; LockPusher adds a geometric object residual while retaining the reference robot forecast; Global permits residual corrections to all state coordinates. The methods share pretrained weights, data, optimization budgets, and adaptation seeds 7, 17, and 27. Checkpoints are selected by 25-step validation object-position error. This comparison tests the contribution of the object residual and the effect of also adapting robot coordinates.

Evaluation protocol. All methods solve the same 1,200 initial-state-goal problems, with a mean initial goal distance of 65 mm. Each planning cycle selects among 128 candidates, executes ten steps, and obtains a new observation. Five cycles give 0.25 s of execution. We report per-coordinate object-position RMSE at step 50, mean final goal distance, and the fraction of trials with a final goal distance of less than 1 mm. The matched full-data comparison is summarized in Table 1.

Results. LockPusher achieves 1.58 mm object-position RMSE, 1.43 mm final goal distance, and 76.53% success. Relative to Global, prediction error decreases by 46.98% and success increases by 11.24 percentage points. Thus, restricting robot-side adaptation can accompany improved object predic-

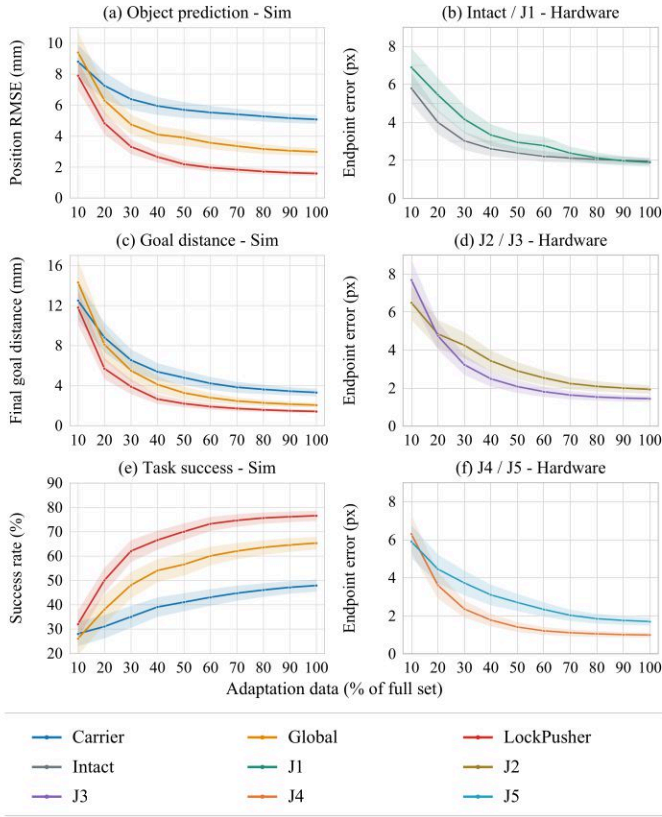


Fig. 4. LockPusher improves simulation prediction and task performance across the tested data budgets (left); hardware endpoint error decreases as adaptation data increase (right). Shading shows mean ± 1 SD over three training seeds. Hardware 100% denotes 100 adaptation trajectories per condition.

tion and closed-loop task performance in this benchmark.

Effect of adaptation-data budget. We evaluate ten adaptation-data fractions from 10% to 100% and retain the same evaluation tasks at each fraction. Figure 4(a,c,e) reports the mean and one standard deviation over three training seeds. LockPusher achieves lower mean object-position RMSE and final goal distance, together with higher mean task success, at every evaluated fraction. Its simulation advantage therefore extends across the tested adaptation budgets.

B. Module analysis and ablations

Object adaptation and reference preservation. We test whether selective readout and independent recurrence retain object corrections while controlling their effect on robot predictions. With fixed checkpoints and paired inputs, the two experiments change the returned coordinates and the rollout feedback, respectively.

Effect of selective readout. Using the same historical checkpoints, we compare full-state intervention and selective readout. Both readouts use the same object prediction, so both retain the 16.74% mean reduction in composite object-state RMSE; they differ in the robot coordinates returned to the planner. Full-state intervention reduces free-state error in 24 combinations and increases it in three, by up to 6.20%. Selective readout retains Carrier’s free-state and pusher errors in all 27 combinations (Table 2).

	Object RMSE reduction (%)	Free worse	Max. free increase (%)	Free/FK match
Carrier	0.00	0/27	0.00	27/27
Full-state	16.74	3/27	6.20	0/27
Selective	16.74	0/27	0.00	27/27

Table 2. Historical selective-readout ablation across 27 combinations. Both readouts retain the same object-error reduction, while selective readout preserves the reference robot prediction in every combination. Free worse counts increased free-state errors; Free/FK match counts unchanged free-state and forward-kinematics pusher errors. Changes are relative to Carrier.

Condition	Object endpoint error	
	mean (px)	Successes/trials
Intact	1.94	3/3
J1	1.88	3/3
J2	1.92	3/3
J3	1.43	3/3
J4	0.97	3/3
J5	1.68	3/3
All	1.64	18/18

Table 3. Real-world results for 40-pixel target displacements. Each success entry gives successful trials out of three attempts. LockPusher completes all tested pushes across the intact arm and five joint-lock conditions; endpoint errors are condition means in pixels.

Effect of independent recurrence. We fix weights and inputs and compare shared-state feedback with independent recurrence on 12 trajectories in each of seven conditions, totaling 84 trajectories. Three checkpoints, seven conditions, and horizons of 10, 25, and 50 steps give 63 evaluation combinations. Independent recurrence preserves the reference robot prediction in all 63 combinations, whereas shared feedback changes it in all 63; object-position and velocity RMSE are identical. Robot error decreases in 60 combinations and increases in three. The historical reset protocol awaits independent physical-validity checks.

C. Real-world experiments

Hardware setup. The hardware platform and recorded intact-arm and joint-lock executions are shown in Figs. 6 and 5; for the deployed controller, we execute three pushes on the intact arm and under each of the J1–J5 locks, for 18 trials. Each target displacement is 40 image pixels, approximately 51.2 mm under the calibration used for this batch. The controller uses the seed-27 deployment weights and evaluates at least 10,000 joint-reference sequences over 50 steps. It executes the selected reference with a 5-degree/s speed limit and replans from a new observation. Repetitions within each condition reuse a preparation candidate set.

Evaluation. Figure 7 illustrates the image-space criterion. A trial succeeds when target error is at most two pixels on each image axis in three distinct, consecutive reliable frames. Object endpoint error is computed separately as the norm of the coordinate-wise median XY error over the final ten frames. Table 3 reports both object endpoint error and task outcomes.

Pushing under joint locks. The intact arm and all five single-joint-lock conditions complete three pushes each, giving

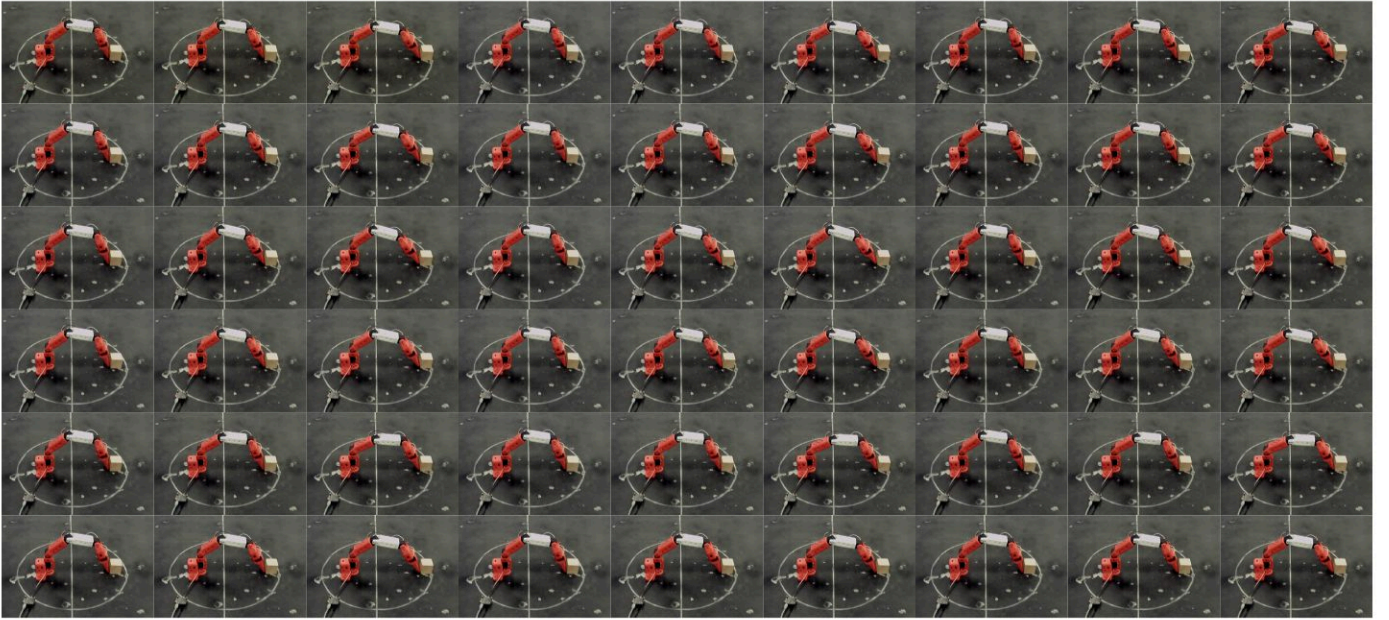


Fig. 5. LockPusher performs nonprehensile pushing with the intact arm and under five single-joint locks. Three recorded executions per condition illustrate its ability to continue object manipulation with different joints immobilized.



Fig. 6. We impose joint locks on the same physical arm and record nonprehensile pushing with wrist and overhead cameras.

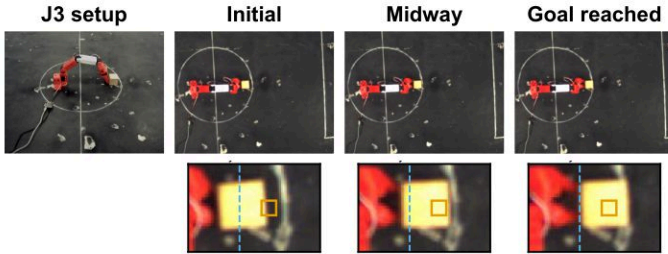


Fig. 7. Under a J3 joint lock, our controller uses visual replanning to push the object toward its goal and confirms arrival through the center-point error in three consecutive frames.

18 successful trials. Mean object endpoint error is approximately 1.64 pixels, with condition means from 0.97 to 1.94 pixels.

The same data-budget study continues on hardware: we collect 100 complete adaptation trajectories for each of the intact-arm and five joint-lock conditions, totaling 600 trajectories. Each fraction uses 10–100 trajectories from its own condition and is evaluated on the same test tasks. Figure 4(b,d,f) reports means and one standard deviation over three training seeds. Mean object endpoint error decreases as adaptation data increase in all six conditions, extending the simulation trend to physical execution.

VI. CONCLUSION AND FUTURE WORK

We presented LockPusher, a world-model framework for nonprehensile pushing under diagnosed joint locks. By separating the reference robot forecast from adapted object dynamics, LockPusher learns contact-response changes while preserving the robot forecast for each candidate action, and uses the adapted object prediction to guide pushing. Simulation shows lower prediction error and higher task success across the evaluated adaptation budgets. Hardware experiments demonstrate pushing with the intact arm and under five single-joint locks. This combination provides an explicit interface for contact planning with motion-restricted robots.

The formulation assumes a known lock diagnosis and a fixed-orientation planar object, making admissible motion and contact geometry explicit. Its central principle defines the scope of adaptation: object response is updated while a designated robot forecast is preserved throughout recurrence. Future work will address object rotation, diverse contact geometries, changing robot dynamics, and constraints that balance robot-prediction updates with planning consistency.

REFERENCES

- [1] C. Finn and S. Levine, “Deep visual foresight for planning robot motion,” in *2017 IEEE Int. Conf. Robotics and Automation (ICRA)*, 2017, pp. 2786–2793.
- [2] M. Bauza, F. R. Hogan, and A. Rodriguez, “A data-efficient approach to precise and controlled pushing,” in *Proc. 2nd Conf. on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 87, 2018, pp. 336–345.
- [3] S. Kim, B. Lim, Y. Lee, and F. C. Park, “SE(2)-equivariant pushing dynamics models for tabletop object manipulations,” in *Proc. 6th Conf. on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 205, 2023, pp. 427–436.
- [4] A. Cully, J. Clune, D. Tarapore, and J.-B. Mouret, “Robots that can adapt like animals,” *Nature*, vol. 521, pp. 503–507, 2015.

- [5] F. Yang, C. Yang, D. Guo, H. Liu, and F. Sun, "Fault-aware robust control via adversarial reinforcement learning," in *2021 IEEE 11th Annual Int. Conf. on CYBER Technology in Automation, Control, and Intelligent Systems (CYBER)*, 2021, pp. 109–115.
- [6] Z. Zhong, S. Golestaneh, and C. Chamzas, "ActivePusher: Active learning and planning with residual physics for nonprehensile manipulation," in *IEEE Int. Conf. Robotics and Automation*, 2026.
- [7] J. Lyu, Z. Li, X. Shi, C. Xu, Y. Wang, and W. He, "DyWA: Dynamics-adaptive world action model for generalizable non-prehensile manipulation," in *Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV)*, 2025, pp. 11 058–11 068.
- [8] A. Kumar, Z. Fu, D. Pathak, and J. Malik, "RMA: Rapid motor adaptation for legged robots," in *Proc. Robotics: Science and Systems (RSS)*, 2021.
- [9] S. M. Richards, N. Azizan, J.-J. Slotine, and M. Pavone, "Adaptive-control-oriented meta-learning for nonlinear systems," in *Robotics: Science and Systems (RSS) XVII*, 2021.
- [10] A. Allevato, E. S. Short, M. Pryor, and A. Thomaz, "TuneNet: One-shot residual tuning for system identification and sim-to-real robot task transfer," in *Proc. Conf. on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 100, 2020, pp. 445–455.
- [11] L. Cong, M. Görner, P. Ruppel, H. Liang, N. Hendrich, and J. Zhang, "Self-adapting recurrent models for object pushing from learning in simulation," in *2020 IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*. IEEE, 2020, pp. 5304–5310.
- [12] H. Gao, Y. Ouyang, and M. Tomizuka, "Online learning in planar pushing with combined prediction model," in *2021 American Control Conference (ACC)*. IEEE, 2021, pp. 2529–2536.
- [13] J. Li, W. S. Lee, and D. Hsu, "Push-net: Deep planar pushing for objects with unknown physical properties," in *Proc. Robotics: Science and Systems (RSS)*, 2018.
- [14] K.-T. Yu, M. Bauza, N. Fazeli, and A. Rodriguez, "More than a million ways to be pushed: A high-fidelity experimental dataset of planar pushing," in *2016 IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*. IEEE, 2016, pp. 30–37.
- [15] J. Zhou, M. T. Mason, R. Paolini, and J. A. Bagnell, "A convex polynomial model for planar sliding mechanics: Theory, application, and experimental validation," *The International Journal of Robotics Research*, vol. 37, no. 2–3, pp. 249–265, 2018.
- [16] A. Ajay, J. Wu, N. Fazeli, M. Bauza, L. P. Kaelbling, J. B. Tenenbaum, and A. Rodriguez, "Augmenting physical simulators with stochastic neural networks: Case study of planar pushing and bouncing," in *2018 IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, 2018, pp. 3066–3073.
- [17] A. Zeng, S. Song, J. Lee, A. Rodriguez, and T. Funkhouser, "Tossing-bot: Learning to throw arbitrary objects with residual physics," *IEEE Transactions on Robotics*, vol. 36, no. 4, pp. 1307–1319, 2020.
- [18] P. Battaglia, R. Pascanu, M. Lai, D. J. Rezende, and K. Kavukcuoglu, "Interaction networks for learning about objects, relations and physics," in *Adv. Neural Inf. Process. Syst.*, vol. 29, 2016.
- [19] A. Sanchez-Gonzalez, J. Godwin, T. Pfaff, R. Ying, J. Leskovec, and P. Battaglia, "Learning to simulate complex physics with graph networks," in *Proc. 37th Int. Conf. on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 119, 2020, pp. 8459–8468.
- [20] K. Zhang, B. Li, K. Hauser, and Y. Li, "AdaptiGraph: Material-adaptive graph-based neural dynamics for robotic manipulation," in *Robotics: Science and Systems (RSS)*, 2024.
- [21] W. Li, H. Zhao, Z. Yu, Y. Du, Q. Zou, R. Hu, and K. Xu, "PIN-WM: Learning Physics-INformed world models for non-prehensile manipulation," in *Robotics: Science and Systems (RSS)*, 2025.
- [22] A. Nagabandi, G. Kahn, R. S. Fearing, and S. Levine, "Neural network dynamics for model-based deep reinforcement learning with model-free fine-tuning," in *2018 IEEE Int. Conf. Robotics and Automation (ICRA)*. IEEE, 2018, pp. 7559–7566.
- [23] R. Mendonca, S. Bahl, and D. Pathak, "Structured world models from human videos," in *Proc. Robotics: Science and Systems (RSS)*, 2023.
- [24] F. R. Hogan and A. Rodriguez, "Reactive planar non-prehensile manipulation with hybrid model predictive control," *The International Journal of Robotics Research*, vol. 39, no. 7, pp. 755–773, 2020.
- [25] G. Wang, K. Ren, and K. Hang, "UNO Push: Unified nonprehensile object pushing via non-parametric estimation and model predictive control," in *2024 IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*. IEEE, 2024, pp. 9893–9900.
- [26] K. Ren, G. Wang, A. S. Morgan, L. E. Kavraki, and K. Hang, "Object-centric kinodynamic planning for nonprehensile robot rearrangement manipulation," *IEEE Transactions on Robotics*, vol. 41, pp. 5761–5780, 2025.
- [27] W. Zhou, B. Jiang, F. Yang, C. Paxton, and D. Held, "HACMan: Learning hybrid actor-critic maps for 6d non-prehensile manipulation," in *Proc. 7th Conf. on Robot Learning*, vol. 229, 2023, pp. 241–265.
- [28] E. Todorov, T. Erez, and Y. Tassa, "MuJoCo: A physics engine for model-based control," in *2012 IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, 2012, pp. 5026–5033.
- [29] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning phrase representations using RNN encoder-decoder for statistical machine translation," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2014, pp. 1724–1734.
- [30] S. Elfwing, E. Uchibe, and K. Doya, "Sigmoid-weighted linear units for neural network function approximation in reinforcement learning," *Neural Networks*, vol. 107, pp. 3–11, 2018.